VANIER
CÉGEP/COLLEGE

Project Planning & Prototyping

J. F. N. Salik,

247-519-VA

Department of Computer Engineering Technology
Vanier College
821 Sainte-Croix Avenue
Saint-Laurent, Québec H4L 3X9

This document was last modified on
Friday 29th May, 2026 at 3:10am.

Title	**Project Planning & Prototyping**
Date	May 29, 2026
Author	J. F. N. Salik, (salikj@vaniercollege.qc.ca)
Identification	247-519-VA
Template	Sparkle Version 1.1 for LaTeX 2ε

VANIER
CÉGEP/COLLEGE

Contents

1 Overview: Prototypes, Products & the Business of Engineering **1**
- 1.1 What You Will Build . . . 1
- 1.2 The Course Map: Two Tracks Over Twelve Weeks . . . 2
- 1.3 From Prototype to Product: The Gap . . . 3
- 1.4 Working with Engineers: What You Own . . . 4
- 1.5 Engineering Businesses by Size . . . 5
- 1.6 Science for Profit: Cost, Time, and Volume . . . 6
- 1.7 How a Prototype Becomes Revenue . . . 6
- Review Questions . . . 10

2 The Prototyping Mindset & the Workflow **13**
- 2.1 Prototype = Answer to a Question, Reducer of Risk . . . 13
- 2.2 The Seven Stages . . . 14
  - 2.2.1 Stage 1: Specification & Risk . . . 14
  - 2.2.2 Stage 2: Scoping & Planning . . . 15
  - 2.2.3 Stage 3: Architecture . . . 16
  - 2.2.4 Stage 4: Subsystem Design . . . 16
  - 2.2.5 Stage 5: Build & Integration . . . 16
  - 2.2.6 Stage 6: Validate & Measure . . . 17
  - 2.2.7 Stage 7: Compile & Economics . . . 17
- 2.3 The Controlled Iteration Loop (Design $\leftrightarrow$ Validate) . . . 18
- 2.4 Prototype Types . . . 18
- 2.5 Fidelity Matched to the Question . . . 19
  - 2.5.1 The Effort–Fidelity Model . . . 19
  - 2.5.2 Running Example: Choosing Fidelity for the Sorter . . . 20
- Review Questions . . . 21
- End-of-Chapter Artifact . . . 22
- Chapter Summary . . . 23

3 Prototype vs. Product & Selective Prototyping **25**
- 3.1 Goals and Standards: Learning vs. Reliability . . . 25
- 3.2 Selective Prototyping: Target High Risk and Uncertainty . . . 27
  - 3.2.1 What "Specify-and-Trust" Actually Means . . . 28
- 3.3 The Cost of Changing a Design Later . . . 28
  - 3.3.1 The Cost-of-Change Heuristic . . . 29
  - 3.3.2 Worked Table . . . 29
  - 3.3.3 Implications for Prototype Targeting . . . 30
- 3.4 Known and Off-the-Shelf Subsystems: Specify and Trust . . . 31
  - 3.4.1 Writing the Requirement . . . 31
  - 3.4.2 Subsystem Classification Table . . . 32
  - 3.4.3 Cost-of-Change Calculation: Two Representative Flaws . . . 32
- Review Questions . . . 35

CHAPTER 1

OVERVIEW: PROTOTYPES, PRODUCTS & THE BUSINESS OF ENGINEERING

Every project begins with a claim: *this idea is worth building.* Your job in Project I is to find out whether that claim is true—cheaply, quickly, and with enough documentation that the answer is defensible. By the end of this course you will have built a working proof-of-concept, assembled the record that shows how you got there, and produced the business case that connects your hands-on work to the real world of engineering firms and markets. This chapter gives you the map. You will see what you are building, why it matters, what separates a prototype from a finished product, and what turns a validated prototype into revenue.

Learning Objectives

1. State what a prototype is for and describe the two deliverables you will produce in Project I.
2. Describe how a proven prototype becomes a product and identify the link between Project I and Project II.
3. Explain your role as a technician working alongside engineers.
4. Explain why industry work is justified by the value it creates, and interpret a negative economic finding correctly.
5. Identify the principal business models that turn a validated prototype into revenue.

1.1 What You Will Build

You will build two things: a **proof-of-concept prototype** and **the record of how you got there.**

The prototype is a working artifact that answers a specific technical and business question. It does not need to be pretty, manufacturable, or certifiable. It needs to

demonstrate that the core idea functions under realistic conditions.

The record is a portfolio of documents—specification, risk register, architecture diagrams, build logs, test data, and cost analysis—that accumulate as you work. You do not write the portfolio at the end. You assemble it from artifacts you produce at each stage. If you work that way, the portfolio is nearly complete before you realize you are making one.

Together, the prototype and its record constitute your output from Project I. The record is as important as the hardware: an engineer can reproduce, evaluate, and extend documented work; undocumented work is disposable.

1.2 The Course Map: Two Tracks Over Twelve Weeks

Figure 1.1 shows the full engineering journey from concept to market. Project I occupies the second stage—the prototype—and Project II the third, where the proven concept is developed into a manufacturable product. The arrow between them is not automatic: what crosses it is the documentation you create in Project I.

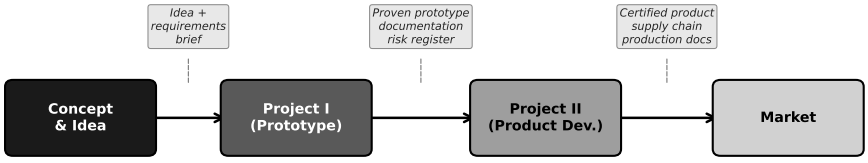

Figure 1.1: The engineering pipeline from concept to market. Project I delivers the proven prototype and its documentation; Project II turns that foundation into a manufacturable product.

The course runs two **parallel tracks**: a theory track and a lab (practice) track. Figure 1.2 shows how they align across the twelve weeks.

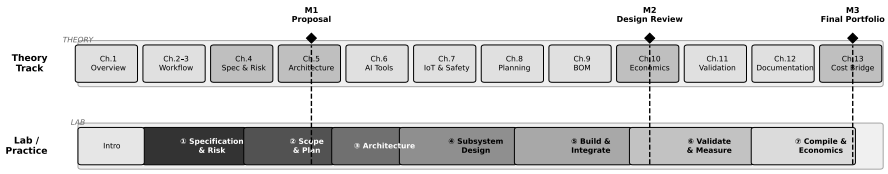

Figure 1.2: The two-track course structure over twelve weeks. The theory track introduces each topic just before the lab requires it; the seven workflow stages unfold across the practice track, with three graded milestones (M1–M3).

The theory track moves through the thirteen chapters of this book at roughly one chapter per week, with Chapters 2 and 3 paired in Week 2. The lab track runs the **seven-stage Prototype Development Cycle**, which you will study in depth in Chapter 2. At the overview level, the seven stages are:

1. **Specification & Risk** — translate the idea into measurable requirements and a risk ranking.

2. **Scoping & Planning** — decide what you will build, set a schedule, and identify long-lead procurement.
3. **Architecture** — decompose the system into subsystems and define their interfaces.
4. **Subsystem Design** — make detailed design decisions for each subsystem.
5. **Build & Integration** — fabricate and assemble; bring subsystems together.
6. **Validate & Measure** — test against the specification; collect data; decide whether to iterate.
7. **Compile & Economics** — assemble the portfolio and analyse the cost and business case.

Three milestones punctuate the semester. **Milestone 1** (Proposal, end of Week 3) delivers your specification and risk register. **Milestone 2** (Design Review, end of Week 9) delivers your architecture, build records, and preliminary cost analysis. **Milestone 3** (Final Portfolio, end of Week 12) delivers the complete project record.

The theory topics are sequenced so that each week's reading lands just as the lab needs it: you study specifications in Week 3, then immediately write your own in the lab. This is not a coincidence—it is the design of the course.

1.3 From Prototype to Product: The Gap

A prototype and a product solve different problems. A **prototype** answers a question about technical feasibility and business viability. A **product** is an artifact designed to be made reliably, in volume, at a cost that leaves margin, by people who were not involved in the original design, and supported over a service life that may span years.

The gap between them is real and wide. A successful prototype does not automatically become a product. The following properties must be designed in—not bolted on afterward:

Manufacturability. Can the design be made in volume using available processes, tooling, and supply chains? Hand-soldered one-of-a-kind assemblies are not manufacturable at scale.

Reliability. Does the design meet a minimum operating life under realistic field conditions? A prototype that runs for an hour in a lab may fail within a week in the field due to thermal cycling, vibration, or humidity.

Cost at volume. Is the per-unit cost, at the production volumes the business requires, low enough to leave an acceptable margin? Chapter 10 develops the cost model in full.

Certification. Does the design meet applicable electrical safety, radio emissions, and (for IoT and robotics) functional safety standards? Certification is a legal prerequisite in most markets, not an optional finishing step.

Supportability. Can the product be diagnosed and repaired by field technicians? Are replacement parts available over the product lifetime? Is the documentation sufficient for a technician who was not present at the original build?

None of these properties is addressed in Project I. That is not a failure; it is appropriate scope management. Your job in Project I is to answer the central technical and business question as quickly and cheaply as possible. Project II then takes that answer as its starting point and builds the full product around it. What crosses the boundary is your documentation: the specification, the architecture, the risk register, the validated design decisions.

1.3.0.0.1 In Practice. A team building a wearable health monitor completes a functional prototype in twelve weeks: it reads heart rate, displays data on a small screen, and transmits to a phone app. The team is pleased—and surprised, six months later, to learn that the product redesign for manufacturing took an additional eighteen months. The enclosure needed injection-moulding tooling. The radio module required FCC and IC certification. The battery management circuit had to be redesigned for the thermal envelope of a wearable. The prototype proved the concept; the product required a separate, larger effort. Neither effort was wasted.

1.4 Working with Engineers: What You Own

In industry you will work on a team that includes engineers. Understanding who owns what prevents frustration and makes you more effective.

Engineers own:

- Requirements—what the system must do, expressed as measurable criteria.
- Design accountability—the engineer signs off on the architecture and carries legal responsibility for the design.
- Risk decisions—the call to proceed, stop, or redesign rests with the engineer of record.

You own:

- Building—breadboarding, PCB assembly, mechanical fabrication, 3D printing, cable harnesses.
- Testing—executing the test procedure, collecting and recording data honestly, flagging anomalies.
- Documenting—keeping the build log current, maintaining the as-built BOM, photographing assembly steps.
- Procuring—sourcing components, getting quotes, placing purchase orders, receiving and inspecting parts.

- Troubleshooting—diagnosing faults at the bench level.

This boundary is not about status; it reflects training and accountability. A technician who takes over design decisions without the engineer's sign-off creates liability for everyone. A technician who executes, documents, and communicates problems clearly makes the engineer's job tractable. Your hands-on judgment—knowing that a solder joint looks wrong, that a component is getting too hot to touch, that the measured current does not match the schematic—is irreplaceable. Engineers who have lost touch with the bench depend on you for that judgment.

1.5 Engineering Businesses by Size

The same prototype, handed to three firms of different sizes, will be handled very differently. Figure 1.3 illustrates how process formality, documentation weight, and approval layers scale with firm size.

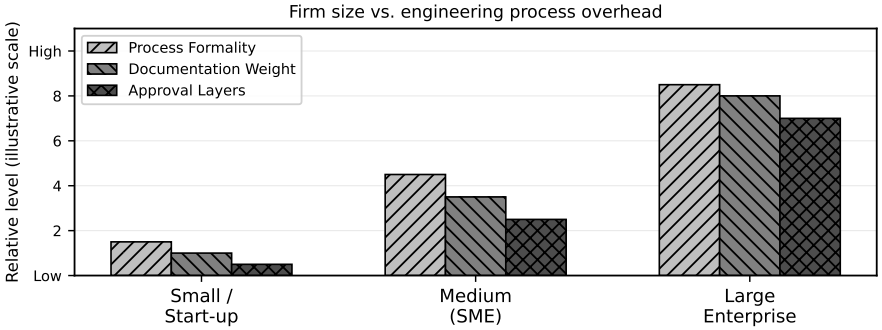

Figure 1.3: Firm-size spectrum: process formality, documentation weight, and approval layers all increase substantially from small start-ups to large enterprises. The prototype development approach must match the firm context.

Small firms and start-ups (under ≈20 people) move fast and tolerate ambiguity. A single engineer may own the entire design. Documentation is minimal—often just enough to reproduce the build. The prototype is frequently made by the same people who conceived it. Speed matters more than process. The risk is that institutional knowledge lives in people's heads and disappears when they leave.

Medium firms (SMEs) (20–500 people) have begun to institutionalise. There are defined roles, some formal review gates, and more documentation requirements. A design change needs sign-off from more than one person. The prototype team is likely distinct from the product team, so documentation quality directly affects the handoff.

Large enterprises (500+ people) operate under heavy process—ISO standards, stage-gate reviews, change control boards, formal test plans, and full configuration management. A prototype at a large firm may require months of internal approval before the first component is ordered. Documentation must be complete, traceable, and auditable.

None of these contexts is inherently better. A start-up that adopts enterprise process collapses under bureaucratic overhead. A large firm that abandons process

loses traceability and produces unrepeatable results. Your job is to recognise the context you are in and calibrate your documentation effort accordingly.

1.5.0.0.1 In Practice. A three-person IoT start-up builds a temperature monitoring module in four weeks: breadboard, firmware, cloud logging, mobile app. The "documentation" is a shared folder with photos and a text file. Two years later, the same company has grown to forty people. A new hire tries to reproduce the original circuit for a customer request. The photos are there; the design rationale is not. It takes three weeks to reverse-engineer the component selection. The original four weeks of work left a four-week debt that compounded with every new hire.

1.6 Science for Profit: Cost, Time, and Volume

Engineering work is not science for its own sake—it is science directed at creating value. This does not diminish the intellectual content; it sharpens the questions. In a commercial context, every technical decision carries an economic consequence, and the three most important constraints are **cost**, **time**, and **volume**.

Cost constrains what you can build and at what margin. A solution that is technically elegant but costs twice the target variable cost is not a viable product.

Time constrains competitive advantage. A prototype finished six months late may miss a market window entirely. Time is the one resource you cannot recover.

Volume determines the economic structure. At low volumes, fixed costs (tooling, engineering, certification) dominate the per-unit cost. At high volumes, variable costs dominate. Chapter 10 develops this model formally; here, it is enough to recognise that the prototype's per-unit cost—often assembled by hand from individual components—tells you nothing about what the product will cost at 1,000 units.

A critical professional skill is the ability to interpret a negative economic finding correctly. Suppose your prototype proves that the design works but the cost analysis shows that no viable margin exists at any realistic production volume. This is not failure—it is the most valuable result the prototype could have produced. You have saved the firm the cost of a full product development effort on an unviable design. Document the finding clearly, state the assumptions, and let the engineering team decide whether to pivot or stop. A prototype that says "no" is doing exactly what a prototype is for.

1.7 How a Prototype Becomes Revenue

Validating a prototype is not the end of the story. The business question that drives the prototype is always, ultimately: how does this create value that someone will pay for? Figure 1.4 maps the major revenue models available once a prototype has been validated.

Product sale (direct). The most familiar model: manufacture a unit, sell it for

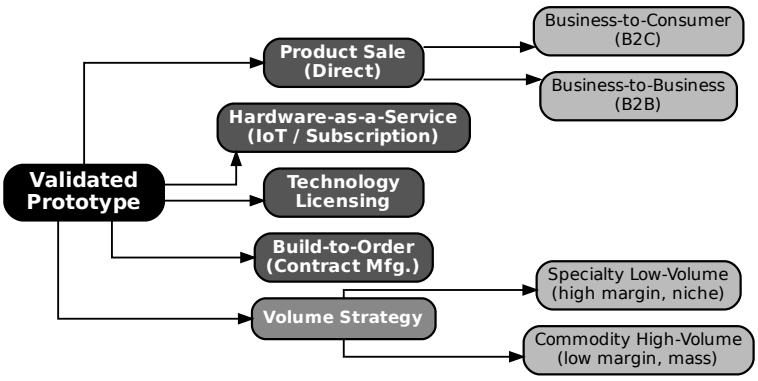

Figure 1.4: Revenue models reachable from a validated prototype. The choice of model shapes what the prototype must prove and what the product development effort must deliver.

more than it costs to make and sell. The two dominant sub-types differ in who the customer is. In a **business-to-consumer (B2C)** model, you sell to individuals; volume is potentially high but margins are typically lower and customer acquisition is expensive. In a **business-to-business (B2B)** model, you sell to companies; volumes are lower but unit prices and margins are generally higher, and relationships carry more weight than brand.

Hardware-as-a-Service (HaaS / IoT subscription). The customer pays an upfront hardware fee and a recurring subscription for connectivity, data processing, and support. The prototype must prove both the hardware function *and* the cloud-connectivity architecture. This model creates recurring revenue and customer lock-in but requires ongoing operational cost for the cloud backend. It is common for IoT devices that produce data the customer values continuously.

Technology licensing. The firm does not manufacture; it licenses the design, firmware, or algorithm to other manufacturers. The prototype proves that the intellectual property is real and valuable. Licensing requires strong IP documentation and often patents.

Build-to-order (contract manufacturing). The firm builds custom units against specific customer purchase orders. Volume is low per customer; the customer drives the specification. Margins can be high because the customer is paying for customisation. The prototype is typically a demonstration to win the contract.

Volume strategy: specialty vs. commodity. Within product sale, the firm chooses where on the volume spectrum to compete. **Specialty low-volume** production targets niche markets with high unit prices and high margins; low volume means less pressure on tooling costs but fewer units over which to amortise fixed costs. **Commodity high-volume** production targets large markets with low unit prices and thin margins; every cent of variable cost matters, and design-for-manufacture is mandatory from the first design decision.

The choice of revenue model is made before the prototype is designed, not after. It shapes what the prototype must prove, what fidelity is needed, and what

documentation must be produced. A HaaS prototype that lacks a working cloud connection has not answered the central business question. A licensing prototype with no IP documentation has nothing to license.

Worked Example: The Sorting Subsystem Business Case

Throughout this book, every chapter works through the same running example: a **connected IoT robotic sorting subsystem**—a sensor-guided actuator that classifies items on a bench conveyor, diverts them to the correct output lane, reports status over a network, and is designed as a module within a larger automation product.

Firm context. The team is a twelve-person engineering start-up that has identified a market in light-industrial sorting for food processing and pharmaceutical packaging. The firm is too small for enterprise process but large enough to have separate engineering and operations functions. Documentation weight is medium: enough to enable reproduction by another team member, not enough to require formal configuration management.

Business model. The chosen model is **Hardware-as-a-Service**: customers purchase the sorter module at a hardware price and pay an annual subscription for cloud-hosted analytics, remote monitoring, and firmware updates. This model requires the prototype to prove two things: (1) the mechanical and electronic sorter works reliably, and (2) the cloud connectivity architecture is sound and can sustain the required data throughput.

What the prototype must answer. The central question is: *Can a sub-$500 CAD bill-of-materials sorter module achieve $\geq$95% sort accuracy at 6 items/min while maintaining a reliable MQTT connection over a standard Wi-Fi network?* If yes, the hardware is viable and the HaaS model can proceed. If no, the firm needs to know whether the failure is in the mechanical design, the sensing, the firmware, or the network layer—so the portfolio must be structured to show that.

Margin calculation. Suppose the firm targets a hardware unit price of $p = \$1{,}200$ CAD and estimates the variable cost of a production unit (materials + direct labour at volume) at $c_{\text{var}} = \$480$ CAD. (These are early estimates; Chapter 10 will refine them using the full cost model.)

The unit margin is:

$$m = p - c_{\text{var}} = \$1{,}200 - \$480 = \$720 \text{ CAD/unit}. \tag{1.1}$$

The gross margin fraction is:

$$\text{GM} = \frac{p - c_{\text{var}}}{p} = \frac{\$720}{\$1{,}200} = 0.60 \quad (60\%). \tag{1.2}$$

A 60% hardware gross margin is strong for an electromechanical module. It must, however, sustain both the hardware warranty cost and the ongoing cost of the cloud subscription infrastructure. Chapter 10 will introduce the full unit-cost model, $C_{\text{unit}}(N) = C_{\text{fixed}}/N + c_{\text{var}}$, which shows how the fixed non-recurring costs (tooling, firmware development, certification) dilute this margin at low production volumes.

Project context brief (your first portfolio artifact).

- *Project:* IoT robotic sorting subsystem module.
- *Business model:* Hardware-as-a-Service; hardware sale + annual cloud subscription.
- *Firm context:* 12-person engineering start-up; medium documentation weight.
- *Business question the prototype must answer:* Can the module achieve $\geq$95% sort accuracy at 6 items/min with a reliable cloud connection, at a BOM cost below $500 CAD?

Try It Yourself

A micro-robotics start-up is developing a vision-guided pick-and-place module. They plan to sell it directly to electronics assembly firms (B2B product sale). Their target unit price is $p = \$950$ CAD and their estimated variable cost is $c_{\text{var}} = \$285$ CAD.

1. Compute the unit margin $m = p - c_{\text{var}}$.
2. Compute the gross margin fraction $\text{GM} = (p - c_{\text{var}})/p$ and express it as a percentage.
3. Using Figure 1.4, identify which revenue model applies and state one thing the prototype must prove to make that model viable.

(Answers not provided — work through the arithmetic before moving on.)

1.7.0.0.1 Common Pitfalls. Technically impressive $\neq$ good prototype. A prototype that demonstrates six different features but answers none of the business questions clearly is not a good prototype. Novelty and complexity are not success criteria. The success criterion is: *did you answer the question?*

"**It works**" $\neq$ "**the business case is proven.**" A prototype that demonstrates function but carries a BOM cost of $1,400 CAD against a $1,200 target price is not a viable product, regardless of how well it works. The economic analysis is not an afterthought—it is half of the prototype's output.

Underrating the technician's judgment. Engineers rely on your observation that the actuator is running hotter than expected, that the network connection drops in the presence of the motor drive, or that a component you received does not match the datasheet. These observations, documented and communicated promptly, are the inputs that prevent costly mistakes. Do not wait to be asked.

Reading a negative economic finding as failure. If the cost analysis shows that the HaaS model is not viable at any realistic volume, that result should be reported clearly, not buried or explained away. The firm can decide to pivot—to a different revenue model, a different component set, or a different market. It cannot decide anything if the analysis is suppressed.

Review Questions

1. **(Conceptual)** A product must satisfy five properties that a prototype does not. List all five and write one sentence explaining what each requires in practice.
2. **(Conceptual)** The chapter states that "a prototype that returns a negative economic result is not a failure." Explain this claim in your own words. What should the team do with the finding, and why is suppressing it dangerous?
3. **(Applied — calculation)** A start-up prices a wireless sensor node at $p = \$340$ CAD and initially estimates $c_{\text{var}} = \$136$ CAD.
   (a) Compute m and GM%.
   (b) After a detailed BOM review, the true variable cost is $c_{\text{var}} = \$204$ CAD. Recompute m and GM%.
   (c) By how many percentage points did the gross margin change? Comment on whether the product remains financially attractive.
4. **(Applied — classification)** A company has developed a machine-learning algorithm that identifies crop disease from smartphone photos. They plan to license it to agricultural equipment manufacturers. Using Figure 1.4 and the definitions in §1.7, identify the revenue model and state two things the prototype must demonstrate to support that model.
5. **(Applied — multi-step)** A proof-of-concept prototype costs $2,400 CAD to build. The team estimates that the eventual product will sell at $p = \$600$ CAD with $c_{\text{var}} = \$180$ CAD.
   (a) Compute m and GM%.
   (b) The $2,400 build cost is *not* c_{var}. Name the cost category it belongs to, and explain why it does not appear in the margin formula at this stage. (*Hint: Chapter 10 will introduce the full cost model.*)

End-of-Chapter Artifact: Project Context Brief

By the end of Week 1, you will produce a one-page **project context brief** for your own project. It is the first document in your portfolio and frames everything that follows. It must contain:

1. A description of the project (one paragraph, no jargon).
2. The chosen business model, with a one-sentence justification.
3. The firm context (size, documentation standard, your role).
4. The single business question the prototype must help answer.
5. A preliminary margin estimate: state p, c_{var}, compute m and GM%, and note the assumptions behind c_{var}.

Keep the brief to one page. Its purpose is not comprehensiveness but clarity: after reading it, a second team member should be able to say whether the project is technically and commercially plausible at a first glance.

Summary

Project I delivers two things: a proof-of-concept prototype and the portfolio that documents how you built and tested it. The prototype's job is to answer one specific technical and business question—not to be a miniature product. Between Project I and Project II lies a deliberate gap: manufacturability, reliability, cost at volume, certification, and supportability must all be designed into the product in Project II, using the prototype's answers as a foundation.

As a technician, you own the build, the test, the documentation, and the procurement. Your hands-on judgment is the most direct input the engineering team has to reality.

Firms handle prototypes differently depending on their size: start-ups move fast with minimal process; enterprises move carefully with heavy documentation. Calibrate your effort to the context.

The economic constraints—cost, time, and volume—are design variables, not external obstacles. A prototype that returns a negative economic result is not a failure; it is evidence. The revenue model chosen before the prototype is built shapes everything: what the prototype must prove, what fidelity is required, and what documentation must be produced.

Up next: Chapter 2 introduces the **seven-stage Prototype Development Cycle** in full. You will see exactly how each stage builds on the previous one, why prototypes answer questions rather than produce things, and how to choose the fidelity level that matches the question at hand—without wasting effort on features the question does not require.

CHAPTER 2

THE PROTOTYPING MINDSET & THE WORKFLOW

Every prototype you build costs time, materials, and opportunity. Build the wrong one — too early, too polished, or targeting the wrong question — and you burn weeks that a one-day breadboard experiment would have saved. This chapter gives you the reasoning scaffold that prevents that waste: the **Prototype Development Cycle**, a seven-stage process that turns an ambiguous project brief into a disciplined sequence of validated decisions. By the end of the chapter you will be able to plan a prototype, defend your fidelity choice with arithmetic, and hand a concise question statement to any reviewer, client, or collaborator.

Learning Objectives. After working through this chapter you will be able to:

1. Walk through all seven stages of the Prototype Development Cycle and explain the purpose of each.
2. Define a prototype by the question it answers and the risk it reduces.
3. Apply the controlled Design $\leftrightarrow$ Validate iteration loop correctly.
4. Select the appropriate prototype type for a given engineering question.
5. Choose fidelity deliberately based on the question, not habit.

2.1 Prototype = Answer to a Question, Reducer of Risk

A prototype is not a small version of the final product. It is not a demonstration for a trade show, a proof that your team is making progress, or practice for the real build. A prototype is a **minimum sufficient artifact** constructed to answer one specific technical or business question and to retire a corresponding risk.

Write the question down before you pick up a soldering iron. If you cannot write it in a single sentence, you do not yet know what you are building. Consider the difference between these two framings:

- *Vague:* "We need to build a prototype of the sorter."

- *Precise:* "Can a sub-$500 CAD BOM sorter module achieve ≥95 % sort accuracy at 6 items/min with reliable MQTT connectivity over standard Wi-Fi?"

The precise framing tells you exactly what to build (a low-cost sorter mechanism with Wi-Fi MQTT), what to measure (accuracy at a fixed throughput rate), and what threshold constitutes a "yes" answer. It also tells you what *not* to build: an injection-moulded housing, CE-marked power supply, or production PCB layout are irrelevant until the accuracy and connectivity question is answered.

Risk in this context is the probability that an unverified assumption collapses the project. Every assumption you have not tested is a risk. A prototype converts an assumption into a measurement, and a measurement into a decision. Figure 2.1 shows how even a low-fidelity prototype delivers substantial intrinsic value — knowledge and risk reduction — long before artifact quality approaches production standards.

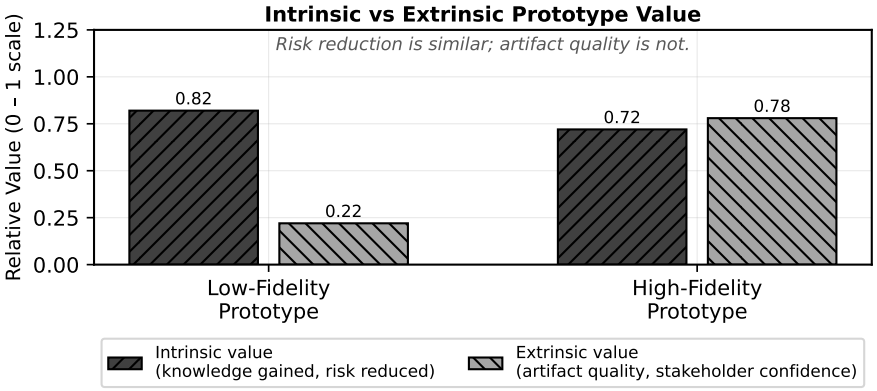

Figure 2.1: Intrinsic value (knowledge, risk reduction) is substantial even at low fidelity; extrinsic value (artifact quality, stakeholder confidence) grows mainly at high fidelity. Build to the fidelity the question requires, not the fidelity that impresses.

The discipline of framing every build as an answer to a question keeps your team aligned and prevents the single most common prototyping failure: building features that are not yet questions.

2.2 The Seven Stages

Figure 2.2 shows the full Prototype Development Cycle. The seven stages run left to right; Stages 4 through 6 form the core build-test loop that iterates until the central question is answered. The loop is controlled — you exit it on a criterion, not when you run out of time.

2.2.1 Stage 1: Specification & Risk

Stage 1 transforms the project brief into a structured set of **specifications** and a ranked list of **risks**. You are not designing anything yet. You are reading, asking questions, and writing.

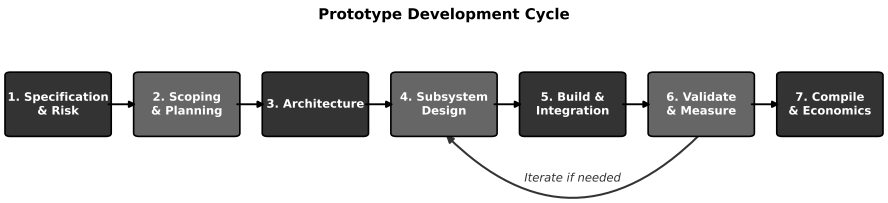

Figure 2.2: The seven-stage Prototype Development Cycle with the Design ↔ Validate iteration loop. Stages 4 through 6 form the core build-test loop; iteration is controlled, not ad-hoc.

A specification at this stage has four components:

1. *Functional requirements* — what the system must do ("sort at $\geq$6 items/min").
2. *Performance bounds* — measurable thresholds ("$\geq$95% accuracy", "MQTT round-trip $\leq$200 ms").
3. *Constraints* — non-negotiable limits ("BOM cost $\leq$$500 CAD", "fits on a 600 mm bench conveyor").
4. *Assumptions* — things believed to be true but not yet measured.

Every assumption in item 4 is a risk candidate. You rank risks by the product of probability and impact, and you identify which prototype stage will retire each one. By the end of Stage 1 you have a written specification document and a risk register. These are the two artifacts that every subsequent stage references.

2.2.1.0.1 In Practice. A one-hour specification session with your full team at the whiteboard saves more time than a week of uncoordinated building. Force every team member to write at least one assumption. Assumptions that no one writes down are the ones that kill projects in Week 10.

2.2.2 Stage 2: Scoping & Planning

Stage 2 converts the risk register into a prototype plan. You decide: which risks must be retired by this prototype, what fidelity level is sufficient to retire them, how long the build will take, and what success looks like.

The key output is a **prototype plan** with:

- The central question (one sentence).
- The chosen fidelity level and justification.
- A work-breakdown with task ownership and duration.
- Explicit pass/fail criteria for the validation step.

Scoping means saying "not this prototype." If the question is about sort accuracy, the housing material is out of scope. Resist the urge to answer every question

at once; you will get a chance in the next milestone. A well-scoped prototype plan is a document your supervisor can review in ten minutes and approve or correct before your team spends any build time.

2.2.3 Stage 3: Architecture

Stage 3 partitions the system into **subsystems** with defined interfaces. You are not selecting components yet; you are drawing boundaries.

A useful architecture diagram shows:

- Each subsystem as a labelled block.
- Every signal and power interface as an arrow with direction and type (digital, analog, mechanical, protocol name).
- The measurement points where validation data will be collected.

The architecture is the contract between team members. If two people are building adjacent subsystems without an agreed interface, integration will fail. In Stage 3 you negotiate and document those contracts before anyone starts detailed design.

Architecture decisions also constrain which subsystem risks are separable. If sort accuracy depends on both the sensor subsystem and the conveyor speed controller, you cannot test one without the other — and your plan must reflect that coupling.

2.2.4 Stage 4: Subsystem Design

Stage 4 is detailed design: component selection, schematic capture, firmware architecture, mechanical part design, and BOM costing. This is where your prior coursework in PCB design, embedded programming, and mechanical CAD becomes the primary activity.

In a prototype context, Stage 4 output does not need to be production-ready. A breadboard schematic is a legitimate Stage 4 artifact if it enables the Stage 5 build to proceed and the Stage 6 measurement to be valid. Keep every design decision traceable to a specification line or a risk item. If you cannot trace it, question whether you need it.

Stage 4 is the entry point to the Design $\leftrightarrow$ Validate loop described in Section 2.3. You design at the subsystem level, build the subsystem, measure it, and return to design if the measurement fails — before integrating with adjacent subsystems.

2.2.5 Stage 5: Build & Integration

Stage 5 is fabrication and assembly: soldering, 3D printing, firmware flashing, wiring, and bringing subsystems together at their defined interfaces.

Integration is a distinct activity from subsystem build. Subsystems that each pass their individual tests routinely fail at integration because interface assumptions were wrong. Run integration in a defined order: bring up the simplest subsystem first, verify its standalone behaviour, then attach the next subsystem and re-verify the combined behaviour before adding the third.

Document every deviation from the Stage 4 design as it happens. A post-hoc reconstruction from memory is inaccurate and defeats the purpose of having a design stage at all.

2.2.5.0.1 In Practice. Keep a shared build log — a shared document or a lab notebook photographed daily — recording what was built, what changed from the design, and any anomalies observed. The build log is the evidence that your Stage 6 validation measurements are trustworthy.

2.2.6 Stage 6: Validate & Measure

Stage 6 is where you answer the question you wrote in Stage 2. **Validation** is a structured measurement campaign against the pass/fail criteria defined before you built anything.

A valid measurement has:

- A defined test procedure that a second engineer can reproduce.
- Sufficient repetitions to estimate uncertainty (minimum 10 trials for a binary outcome like "sort correct/incorrect").
- Documented environmental conditions (supply voltage, temperature, Wi-Fi signal strength, conveyor belt load).
- A recorded outcome: pass, fail, or inconclusive with reason.

If the outcome is "pass," you exit the loop and proceed to Stage 7. If the outcome is "fail," you return to Stage 4 with a specific design change hypothesis, not a vague intention to "try something different." This controlled return is what distinguishes disciplined iteration from thrashing.

2.2.7 Stage 7: Compile & Economics

Stage 7 closes the prototype milestone. You compile the evidence: test results, BOM cost actuals, build time log, and a comparison of achieved performance against specification. You then assess the economic implications.

For a commercial project, Stage 7 includes a preliminary **unit economics** analysis: does the validated BOM cost, at the production volume assumed in the business model, support the target margin? For the running example — a 12-person start-up with a Hardware-as-a-Service model, price $p = \$1{,}200$ CAD, and variable cost $c_{\text{var}} = \$480$ CAD — the contribution margin per unit is:

$$m = p - c_{\text{var}} = \$1{,}200 - \$480 = \$720 \text{ CAD} \tag{2.1}$$

If the BOM comes in above $500 CAD target, Stage 7 flags that the economics are compromised before the team invests in Milestone 2 tooling. Stage 7 also produces the **go/no-go recommendation** for the next milestone and a list of risks that remain open.

2.3 The Controlled Iteration Loop (Design ↔ Validate)

Stages 4, 5, and 6 form a loop. The loop is necessary because prototypes reveal information that design alone cannot predict: a sensor saturates at lower ambient light than the datasheet implied, firmware interrupt latency is 40% higher than estimated, a 3D-printed bracket flexes under motor torque. Iteration is not a sign of poor planning; it is the mechanism by which prototyping generates knowledge.

What must be controlled is *how* you iterate.

Uncontrolled iteration looks like this: the test fails, someone changes three things simultaneously, the next test passes, and no one knows which change fixed the problem. This is common; it is also useless as engineering knowledge, because you cannot reproduce the fix reliably or explain it to a manufacturing partner.

Controlled iteration imposes two rules:

1. *One variable at a time.* When a measurement fails, form a hypothesis about the cause, change exactly one design element, and re-measure. If you must change two things to make the system testable, document the dependency explicitly.
2. *Exit on a criterion, not on a deadline.* Define in Stage 2 what "good enough" means. Exit the loop when the measurement meets that criterion. Do not exit because the schedule says so without recording which criteria remain unmet.

A useful framing: every pass through the loop should reduce uncertainty by answering a smaller sub-question. If you cannot name the sub-question before you change something, you are not iterating — you are guessing.

2.3.0.0.1 In Practice. Limit yourself to three full loop passes before calling a team checkpoint. If after three passes the central question is not answered, the problem is likely in the scope, the architecture, or the specification — not in the build. Step back to Stage 2 or Stage 3 rather than continuing to iterate at Stage 4.

2.4 Prototype Types

Not all prototypes serve the same purpose. Selecting the right type for your question prevents over-building and keeps the team focused. The four primary types used in engineering product development are:

Proof-of-Concept (PoC) Answers a binary feasibility question: can the physics work? Appearance is irrelevant. A PoC uses whatever materials are fastest to assem-

ble — breadboards, hook-up wire, 3D-printed brackets, off-the-shelf modules. Target fidelity: Level 1 or 2.

Functional prototype Demonstrates that the system performs its core function in representative conditions. Appearance may still be a mock-up. Firmware is real but not hardened. Used to answer "does it work under realistic operating conditions?" Target fidelity: Level 3.

Looks-like/works-like prototype Combines representative form and full function. Housing geometry, interface locations, and weight approach final product. Used to answer "is the user experience acceptable?" and "can we hit the cost target?" Target fidelity: Level 4.

Pre-production prototype Built from production-intent processes: PCB from the chosen contract manufacturer, housing from production tooling, firmware at release candidate version. Answers: "will this survive the factory and the field?" Target fidelity: Level 5.

Choosing a looks-like/works-like prototype to answer a feasibility question is the most common and most expensive mistake in early-stage product development. Match the type to the question first; fidelity follows from type.

2.5 Fidelity Matched to the Question

Fidelity is the degree to which a prototype resembles and performs like the final product. Higher fidelity costs more. The cost growth is not linear.

Figure 2.3 shows relative effort plotted against fidelity level. The relationship is approximately exponential.

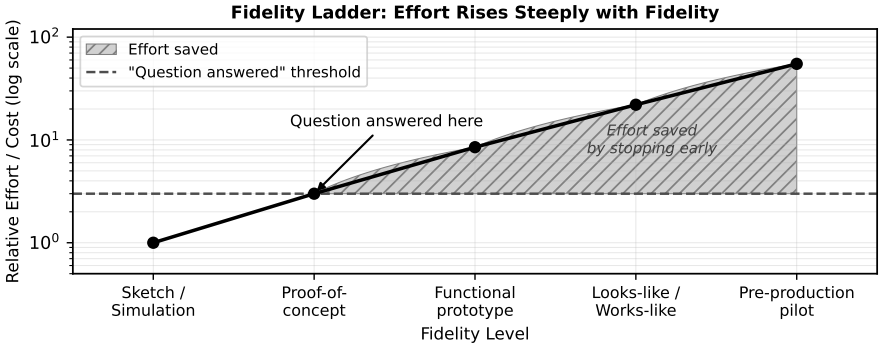

Figure 2.3: Relative effort rises steeply with fidelity level. A proof-of-concept (Level 2) answers many questions at a fraction of the cost of a looks-like/works-like prototype (Level 4).

2.5.1 The Effort–Fidelity Model

Let $f \in \{1, 2, 3, 4, 5\}$ be the fidelity level, where $f = 1$ is a hand sketch or simulation and $f = 5$ is a pre-production prototype built with production-intent processes. Define **relative effort** $E(f)$ as:

$$E(f) = E_0 \cdot k^f, \qquad k > 1 \tag{2.2}$$

where E_0 is a baseline effort constant and k is the per-level effort multiplier. The model captures the empirical observation that each fidelity step requires roughly k times the total effort of the step below it, because higher fidelity demands tighter tolerances, more sophisticated tooling, more thorough testing, and more documentation.

2.5.1.1 Effort Table

Set $E_0 = 1$ and $k = 2.5$ as representative values. Table 2.1 shows the resulting effort at each level.

Table 2.1: Relative effort $E(f) = 2.5^f$ for fidelity levels 1 through 5.

Level f	**Type**	**Relative Effort** $E(f)$
1	Sketch / simulation	$2.5^1 = 2.5$
2	Proof-of-concept	$2.5^2 = 6.25$
3	Functional prototype	$2.5^3 = 15.6$
4	Looks-like/works-like	$2.5^4 = 39.1$
5	Pre-production	$2.5^5 = 97.7$

The ratio of effort at $f = 5$ relative to $f = 2$ is:

$$\frac{E(5)}{E(2)} = \frac{k^5}{k^2} = k^3 = 2.5^3 = 15.6 \tag{2.3}$$

If your question is answered at $f = 2$, building to $f = 5$ multiplies your effort by $15.6\times$. Equivalently, $\frac{E(5)-E(2)}{E(5)} \approx 93\,\%$ of the total $f = 5$ effort is *spent after the question is already answered*. That is the waste the fidelity discipline prevents.

2.5.2 Running Example: Choosing Fidelity for the Sorter

Return to the central question for the connected IoT robotic sorting subsystem:

> "Can a sub-$500 CAD BOM sorter module achieve $\geq$95 % sort accuracy at 6 items/min with reliable MQTT connectivity over standard Wi-Fi?"

The team is deciding between two prototype options:

Option A — Looks-like/works-like ($f = 4$): Custom PCB with production layout, injection-moulded housing mock-up, full production firmware with over-the-air update support. Estimated build time: 14 weeks.

Option B — Proof-of-concept ($f = 2$): Breadboard electronics, 3D-printed mounting bracket, firmware with sort accuracy event logging, Wi-Fi MQTT connectivity test. Estimated build time: 3 weeks.

Applying the effort model with $k = 2.5$:

$$\frac{E(4)}{E(2)} = k^{4-2} = 2.5^2 = 6.25 \tag{2.4}$$

Option A requires $6.25\times$ the effort of Option B. Now ask: does the central question require $f = 4$? The question asks about sort accuracy, throughput rate, BOM cost, and MQTT connectivity. None of those require an injection-moulded housing or a production PCB. A breadboard can deliver accurate sensor readings; a 3D-printed bracket can hold the actuator at the required geometry; a Wi-Fi module on a breadboard can carry MQTT traffic.

The decision is Option B. If Option B answers "yes," the team proceeds to Milestone 2 with the validated architecture, and Option A becomes the correct investment at that stage. If Option B answers "no" — the sort accuracy is 78 % at 6 items/min, or the MQTT latency is unacceptable on a congested network — the team has lost 3 weeks, not 14. The cost of a wrong hypothesis is bounded by the fidelity choice.

2.5.2.0.1 Common Pitfalls.

- **"Prototype = mini-product."** Building all product features when only one needs to be proved is the most common and most expensive prototyping error. Injection moulding, CE certification, and production firmware are irrelevant at Stage 4 if the question is about sort accuracy. Every feature you build beyond what the question requires is pure waste until the question is answered.
- **"Higher fidelity is always better."** Fidelity costs effort exponentially, as shown in Table 2.1. A higher-fidelity prototype that cannot yet answer the question delivers zero additional information — it only consumes schedule and budget. Match fidelity to the question; resist the instinct to impress.

Review Questions

1. **(Conceptual)** List the seven stages of the Prototype Development Cycle in order. For Stage 1 and Stage 6, state the primary output that stage produces.
2. **(Conceptual)** Distinguish **intrinsic value** from **extrinsic value** in the context of a prototype. Which type does a proof-of-concept maximise, and why does that make it appropriate for answering a central technical question?
3. **(Applied — calculation)** Using $E(f) = E_0 \cdot k^f$ with $E_0 = 1$ and $k = 3.0$:
   (a) Compute $E(f)$ for $f = 1, 2, 3, 4, 5$.
   (b) What is the ratio $E(4)/E(2)$?
   (c) If a question is answered at $f = 2$, what fraction of the Level-4 effort is saved?
4. **(Applied — decision)** A team must verify that a custom MEMS pressure sensor achieves ± 0.5 Pa accuracy when integrated with their MCU at $-20\,^{\circ}$C.

(a) Identify the most appropriate prototype type from §2.4.

(b) State the fidelity level from §2.5 and justify the choice in one sentence.

(c) Identify one risk that would justify moving to a higher fidelity if the initial result is inconclusive.

5. **(Applied — multi-step)** A team estimates 20 weeks to build a pre-production prototype ($f = 5$) at a weekly cost of $3,500 CAD. Using $k = 2.5$, $E_0 = 1$:

   (a) Compute $E(2)$ and $E(5)$.

   (b) What fraction of the 20-week effort corresponds to $f = 2$?

   (c) How many weeks does the PoC take (round to one decimal place)?

   (d) How much money is saved by stopping at $f = 2$ if the question is answered there?

Try It Yourself

A firmware team is deciding how much prototype fidelity to invest in before committing to a custom RTOS port. They estimate the following parameters: $E_0 = 1$, $k = 3.0$.

1. Using $E(f) = E_0 \cdot k^f$, compute the relative effort at each fidelity level $f = 1, 2, 3, 4, 5$. Organise your results in a table.
2. Compute the ratio $E(5)/E(2)$. What does this number mean in practical terms?
3. If the central question can be answered at $f = 2$, what percentage of the Level-5 effort is wasted by building to $f = 5$ instead?

(Answers not provided — work through the arithmetic before moving on.)

End-of-Chapter Artifact

The deliverable for this chapter is a single written paragraph, submitted to your portfolio, stating: (1) the central question your Milestone 1 prototype will answer, (2) the fidelity level you have chosen, and (3) a justification for that choice in terms of the question, the risk it retires, and the effort saved relative to the next higher fidelity level.

Running-Example Statement (model your own on this):

> The Milestone 1 prototype for the IoT robotic sorting subsystem will answer the following central question: *"Can a sub-$500 CAD BOM sorter module achieve ≥95% sort accuracy at 6 items/min with reliable MQTT connectivity over standard Wi-Fi?"* The chosen fidelity is Level 2 (proof-of-concept), implemented as breadboard electronics, a 3D-printed mechanical mounting bracket, and firmware with sort-event data logging

and Wi-Fi MQTT connectivity. This fidelity is sufficient because the question is about sensor accuracy, actuator throughput, and network protocol performance — none of which require a production PCB layout or an enclosure. Advancing to Level 4 at this stage would multiply effort by $6.25\times$ (Equation 2.4) before the feasibility of the core mechanism is established. If the Level 2 prototype answers "yes," Level 4 is the justified next investment; if "no," the team reframes the architecture having lost 3 weeks rather than 14.

This statement feeds directly into the specification document you will develop in Chapter 4. Keep it in your portfolio; you will reference and refine it at each subsequent milestone.

Chapter Summary

A prototype is the minimum sufficient artifact that answers a specific technical or business question and retires a corresponding risk. The Prototype Development Cycle organises that work into seven stages: Stage 1 (Specification & Risk) establishes what must be true and what is assumed; Stage 2 (Scoping & Planning) sets the question, the fidelity, and the success criterion; Stage 3 (Architecture) partitions the system into subsystems with defined interfaces; Stage 4 (Subsystem Design) produces the design artefacts; Stage 5 (Build & Integration) realises and assembles those designs; Stage 6 (Validate & Measure) answers the question against predefined criteria; and Stage 7 (Compile & Economics) closes the milestone with evidence and a go/no-go recommendation.

Stages 4 through 6 form a controlled iteration loop. Each pass tests one hypothesis, changes one variable, and exits on a measured criterion — not on a deadline and not by accumulated intuition.

Fidelity is not a virtue. It is a variable you set to the minimum level that makes the central question answerable. The effort–fidelity relationship is exponential ($E(f) = E_0 \cdot k^f$), so every unnecessary fidelity level you add multiplies your cost by k before you have an answer. For the sorter running example, choosing Level 2 over Level 4 saves $6.25\times$ the effort on a question whose answer is not yet known.

Transition to Chapter 3. Chapter 3 examines the boundary between prototype and product in detail. You will learn to distinguish which subsystems require prototyping and which can be specified directly from validated prior art, and how to apply *selective prototyping* to allocate your build budget to the risks that matter.

CHAPTER 3

PROTOTYPE VS. PRODUCT & SELECTIVE PROTOTYPING

Every engineering team has a finite budget of time, money, and attention. The moment you begin building something — anything — you are spending that budget. The central question of this chapter is not *how* to prototype, but *what* to prototype and *why*. A well-targeted prototype teaches you something you could not learn any other way and reduces the risk of discovering an expensive flaw later. A poorly targeted prototype keeps the team busy, drains the budget, and leaves the real unknowns untouched.

By the end of this chapter you will be able to draw a clear line between the seven subsystems of your IoT sorting system: four demand a hands-on prototype; three do not. The discipline of drawing that line — and defending it — is the core skill this chapter builds.

Learning Objectives

1. Contrast a prototype and a product across goals, standards, and lifecycle obligations.
2. Separate intrinsic value (knowledge, risk reduction) from extrinsic value (artifact quality) in a practical decision.
3. Apply the cost-of-change heuristic to justify front-loading design decisions.
4. Classify subsystems as either "prototype" or "specify-and-trust" using defined criteria.

3.1 Goals and Standards: Learning vs. Reliability

Start with a question most teams skip: what is the object you are building *for*?

A **prototype** is an artifact whose primary purpose is to generate knowledge. You build it to answer a question, reduce uncertainty, or demonstrate that a physical

principle works at the scale and under the conditions that matter for your product. The prototype may be ugly, fragile, hand-wired, held together with cable ties, and powered by a bench supply. None of that matters as long as it answers the question it was built to answer. The moment it does, its job is done.

A **product** is an artifact whose primary purpose is to deliver reliable value to a customer across the full lifecycle of use. It must meet regulatory requirements, survive handling and shipping, operate within specified environmental limits, and be maintainable and repairable by people who were not in the room when it was designed. Reliability, not learning, is the governing criterion.

This distinction shapes every design decision that follows. Consider the two objects side by side across four dimensions.

3.1.0.1 Goals

A prototype's goal is epistemic: you want to know something you do not yet know. Is the ML inference fast enough on the target MCU? Does the servo divert reliably at 6 items per minute under a 200 g load? A product's goal is operational: deliver the specified function to the customer on schedule, within cost, and without failure in the field.

3.1.0.2 Standards

Prototypes are typically exempt from — or at least not yet subject to — formal certification requirements. You do not CE-mark a breadboard. Products must meet all applicable standards in every jurisdiction where they are sold: electrical safety, EMC, RoHS compliance, industry-specific standards for the operating environment. For your sorting subsystem shipped under a Hardware-as-a-Service (HaaS) model, this means your final product must meet the standards of a production floor, not a university lab.

3.1.0.3 Lifecycle Obligations

A prototype has no lifecycle obligation beyond the experiment it supports. When you have your answer, you can dismantle it. A product carries a service life: firmware updates, spare parts, warranty claims, end-of-life disposal. Designing for lifecycle obligations adds cost and complexity that is inappropriate — and wasteful — to impose on a prototype.

3.1.0.4 Failure Modes

Prototype failure is valuable. If the sensor field of view is too narrow and items slide past unclassified, you have learned exactly what you needed to learn, cheaply. Product failure has consequences: warranty claims, customer downtime, reputational damage, potential liability. The standards a product must meet are precisely the standards required to make failure improbable under foreseeable conditions.

Intrinsic value is the knowledge and risk reduction produced by building and testing an artifact, regardless of whether the artifact itself is ever used again. **Extrinsic value** is the value of the artifact as a deliverable — its functional quality, finish, reliability, and compliance.

A prototype optimizes for intrinsic value. A product must optimize for extrinsic value. The practical mistake most teams make is to apply product-level quality expectations to prototype work — spending days on a clean PCB layout before the core algorithm has been validated — or, conversely, to ship a prototype-quality object to a customer and call it a product.

3.1.0.4.1 In Practice. At your twelve-person start-up, the distinction between prototype and product is under constant pressure. Investors want to see polished demos; customers want reliable hardware; engineers want to close tickets. The discipline is to keep the question clear at all times: are we building this to *learn* something, or to *deliver* something? The answer determines the standard you hold it to, the time you spend on it, and when you are done.

3.2 Selective Prototyping: Target High Risk and Uncertainty

Given that a prototype costs time and money, the decision to prototype a subsystem is a resource allocation decision. **Selective prototyping** is the discipline of directing prototype effort only at the subsystems where the cost of remaining uncertain exceeds the cost of building and testing an artifact.

The decision logic is straightforward and can be expressed as three questions.

1. **Is the subsystem novel?** Has your team built and validated this combination of components, firmware, and mechanical arrangement before? If not, you have a knowledge gap that a prototype can close.
2. **Is the subsystem uncertain?** Does the subsystem's performance under your specific conditions depend on factors you cannot fully resolve from datasheets and first principles? Mechanical resonance under dynamic load, ML inference latency on a specific MCU, Wi-Fi packet loss rates in a metal-walled production environment — these are inherently empirical questions.
3. **Is failure high-consequence?** If the subsystem fails or underperforms, does it compromise the core value proposition or create safety, financial, or customer-relationship risk that is disproportionate?

If the answer to any of these questions is "yes," the subsystem is a prototype candidate. If the answer to all three is "no," the subsystem is a candidate for **specify-and-trust**: you select a well-characterized off-the-shelf (OTS) component, confirm it meets your requirements from the datasheet, and integrate it without building a prototype.

Figure 3.1 shows this decision flow as a diagram. Work through it for each subsystem in your design, and record the decision with a one-sentence justification.

The justification is not bureaucratic overhead — it is the record that tells the next engineer (or future you) why a prototype was or was not built, so the decision can be revisited if requirements change.

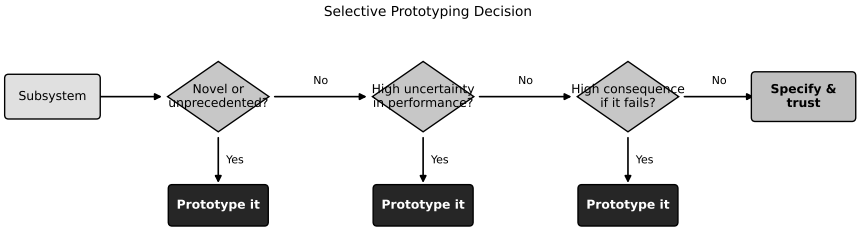

Figure 3.1: Decision flow for classifying a subsystem as prototype or specify-and-trust. Only novel, uncertain, or high-consequence subsystems justify the effort of building a prototype.

3.2.1 What "Specify-and-Trust" Actually Means

Choosing to specify-and-trust is not the same as choosing to ignore. It means you invest your engineering effort in writing a precise requirement — a performance specification with numerical bounds — rather than in building and testing an artifact. You then select a component whose datasheet confirms it meets that specification, document that selection decision, and move on.

For the OTS conveyor belt module in your sorting subsystem, "specify and trust" means you write: "Belt conveyor module shall transport items at 6 ± 0.5 items per minute at 200 g average item mass, with a mean time between failures (MTBF) $\geq 2{,}000$ hours." You select a module whose datasheet meets that spec, log the part number, and integrate it. You do not spend a week running test items over a borrowed belt to confirm what the supplier already guarantees.

The budget you save goes directly to the subsystems that need it: the sensor pipeline, the actuator timing, the firmware, and the MQTT connectivity.

3.2.1.0.1 In Practice. Teams often prototype familiar subsystems first because the work is comfortable and fast. Resist this. The sensor pipeline is the riskiest subsystem in your design, and it is the last thing you want to discover is broken at the integration stage. The decision of *what* to prototype is more important than *how well* any individual prototype is executed.

3.3 The Cost of Changing a Design Later

The strongest argument for front-loading prototype work — for doing it early and systematically — is the cost of change. Design flaws do not get cheaper to fix as a project progresses. They get dramatically more expensive.

3.3.1 The Cost-of-Change Heuristic

Let s denote the **project stage** as an integer:

s	Stage
0	Concept
1	Design
2	Build (Prototype / Pre-production)
3	Production

The **cost-of-change heuristic** states that the cost of fixing a given design flaw at stage s is approximately:

$$C_{\text{change}}(s) \approx C_0 \cdot 10^{s} \tag{3.1}$$

where C_0 is the cost of fixing the same flaw at the Concept stage ($s = 0$). The exponent reflects the compounding difficulty: each stage adds committed procurement, tooling, testing infrastructure, and — in production — physical units that must be reworked or scrapped.

The saving from detecting a flaw at an earlier stage s_1 rather than a later stage s_2 is:

$$\Delta C(s_1, s_2) = C_0 \left(10^{s_2} - 10^{s_1}\right). \tag{3.2}$$

For example, catching a flaw at Design ($s_1 = 1$) rather than Production ($s_2 = 3$): $\Delta C(1, 3) = C_0 \left(10^3 - 10^1\right) = 990\, C_0$.

This heuristic originates in empirical studies of software and hardware projects from the 1970s onward and has been reproduced across aerospace, consumer electronics, and industrial systems. The exact multiplier varies by industry; the order-of-magnitude-per-stage relationship is robust.

3.3.2 Worked Table

Set $C_0 = \$50$ CAD, roughly one engineering hour at a start-up billing rate.

Table 3.1: Cost to fix the same design flaw at each project stage, with $C_0 = \$50$ CAD.

s	Stage	Formula	Cost (CAD)
0	Concept	$\$50 \times 10^0$	\$50
1	Design	$\$50 \times 10^1$	\$500
2	Build	$\$50 \times 10^2$	\$5,000
3	Production	$\$50 \times 10^3$	\$50,000

The savings from finding a flaw at the Design stage rather than at Production is:

$$\begin{aligned}\Delta C &= C_{\text{change}}(3) - C_{\text{change}}(1) \\ &= C_0\left(10^3 - 10^1\right) \\ &= \$50 \times 990 \\ &= \$49{,}500 \end{aligned} \tag{3.3}$$

That is the value created by a prototype that catches one flaw before production begins. At a BOM cost target of $480 CAD per unit and a twelve-person team, finding even a single sensor-integration flaw early pays for many hours of prototype work.

Figure 3.2 illustrates this escalation graphically. The vertical axis is logarithmic; the relationship is linear on that scale, which makes the exponential growth viscerally clear.

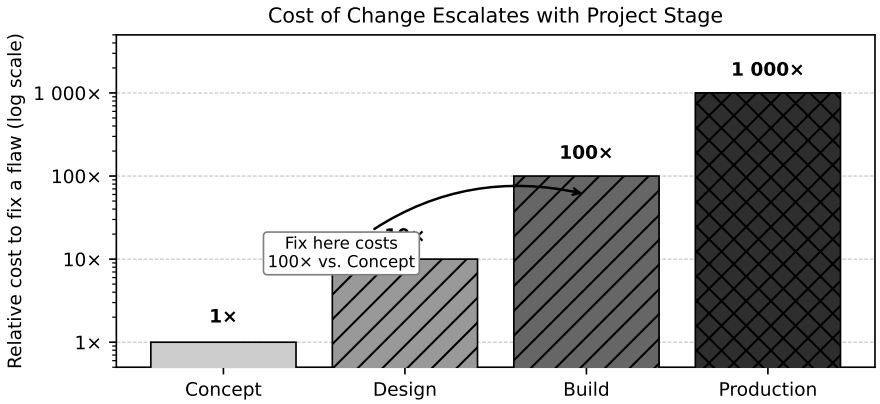

Figure 3.2: Cost of fixing a design flaw escalates by roughly one order of magnitude per project stage. A flaw caught at the Concept stage costs C_0; the same flaw at Production costs $1{,}000\,C_0$.

3.3.3 Implications for Prototype Targeting

Equation (3.1) does not tell you which subsystem to prototype. It tells you *why* the decision matters. Combine it with the three selectivity criteria from Section 3.2: a subsystem that is novel, uncertain, and high-consequence is precisely the subsystem most likely to harbor a flaw that, if undetected until Production, will cost $10^3 C_0$ to fix.

The rational response is to prototype it now, at the cost of $10^0 C_0$ to $10^1 C_0$, and find the flaw while it is still cheap.

3.3.3.0.1 In Practice. A common objection is "we will catch it during integration testing." Integration testing is stage 2 (Build) in the framework above. Finding a sensor field-of-view problem at integration costs $5,000, not $50. The flaw was always findable earlier; the question is whether you built the right prototype to find it.

3.4 Known and Off-the-Shelf Subsystems: Specify and Trust

Not every subsystem is a source of risk. A mature, well-documented off-the-shelf (OTS) module has been designed, tested, and certified by an organization with more relevant experience than your team. The manufacturer has already run thousands of hours of life testing. The datasheet encodes the results of that testing as specifications you can use directly.

Off-the-shelf subsystems are appropriate candidates for specify-and-trust when all of the following hold:

1. The performance parameters you need are *fully specified* in the manufacturer's datasheet or product bulletin.
2. Your operating conditions (load, temperature, supply voltage, duty cycle) fall within the specified operating range with adequate margin.
3. Failure of the subsystem is either detectable before it affects output quality, or its consequence is low enough that reactive replacement is acceptable.
4. The subsystem does not interact in a novel way with other subsystems in your design.

When all four conditions hold, you can write a requirement, confirm the datasheet match, log the decision, and redirect your prototype budget.

3.4.1 Writing the Requirement

"Specify and trust" begins with a clear requirement, not with browsing a catalogue. The requirement defines what the subsystem must do in your system, expressed in terms your team can verify from a datasheet.

For the 5 V / 12 V power supply in the sorting subsystem, a well-formed requirement might be:

> The power supply shall provide 5 V DC $\pm$5% at up to 3 A and 12 V DC $\pm$5% at up to 2 A simultaneously, with output ripple $\leq$ 50 mV peak-to-peak, certified to UL 60950-1 or equivalent, operating at ambient temperatures from 0°C to 50°C.

A switched-mode PSU from any reputable industrial supplier will meet this specification. You confirm the datasheet, log the part number, and move on. No prototype is needed.

3.4.1.0.1 Common Pitfalls.

- **Over-specifying OTS components.** Writing requirements more stringent than your system actually needs narrows the supplier pool unnecessarily and drives up cost. Specify what you need; trust the datasheet to confirm the rest.
- **Under-specifying integration conditions.** An OTS conveyor belt module rated for 200 g loads will fail if your items occasionally reach 350 g. Specify the *actual* worst-case operating conditions, including margin, not the nominal design point.
- **Skipping the datasheet confirmation step.** "Specify and trust" means trust the datasheet, not trust the product name. The confirmation step — reading the datasheet and recording that it meets your requirement — is not optional.

Worked Example: IoT Sorting Subsystem Classification

Apply the selective prototyping framework to the seven subsystems of the IoT robotic sorting system. Recall from Chapter 2 that the central question is: can a sub-$500 BOM sorter achieve $\geq$95% sort accuracy at 6 items per minute with reliable MQTT connectivity? The Level 2 (proof-of-concept) fidelity decision commits you to breadboard electronics, a 3D-printed bracket, and firmware with data logging.

3.4.2 Subsystem Classification Table

Table 3.2 applies the three criteria to each subsystem and records the resulting decision.

The result: **4 subsystems to prototype** (sort sensor, divert actuator, MCU/firmware, MQTT integration); **3 to specify-and-trust** (conveyor belt, PSU, structural frame). This classification feeds directly into the risk register you will build in Chapter 4.

3.4.3 Cost-of-Change Calculation: Two Representative Flaws

Apply Equation (3.1) to two concrete flaws that could reasonably affect the sorting subsystem.

3.4.3.1 Flaw A: Sort Sensor Field of View Too Narrow

The camera is mounted at a fixed height with a fixed lens. During prototype testing you discover that items wider than 80 mm pass partially outside the field of view, causing the ML model to misclassify them.

Detected at the Design stage ($s = 1$):

$$C_{\text{change}}(1) = \$50 \times 10^{1} = \$500 \tag{3.4}$$

Remediation: redesign the bracket to lower the camera by 30 mm, reprint the PLA bracket (cost: 4 hours of engineering time, one print job), rerun the accuracy test. Total cost is roughly one day of work.

Table 3.2: Selective prototyping classification for the IoT robotic sorting subsystem. Four subsystems require prototype effort; three are appropriate for specify-and-trust.

Subsystem	Novel?	Uncertain?	High-cons.?	Decision	Justification
Sort sensor (camera + ML inference)	Yes	Yes	Yes	**Prototype**	Novel ML pipeline; accuracy uncertain; misclassification is the core risk
Divert actuator (servo + gate mechanism)	Partial	Yes	Yes	**Prototype**	Mechanical timing under load is unproven; failure stops sorting
Embedded MCU + firmware	Yes	Yes	Yes	**Prototype**	Custom firmware and sensor integration; timing and power unknown
MQTT / Wi-Fi connectivity	No	Yes	Yes	**Prototype**	Known protocol but real-time reliability on production floor is unproven
Conveyor belt (OTS module)	No	No	No	Specify & trust	Standard industrial conveyor; datasheet specs sufficient
5 V / 12 V power supply (OTS PSU)	No	No	No	Specify & trust	Standard switched-mode PSU; certified component
Structural frame (3D-printed PLA)	No	No	No	Specify & trust	Low-consequence geometry; standard material; no structural certification required

Detected at the Production stage ($s = 3$):

$$C_{\text{change}}(3) = \$50 \times 10^3 = \$50{,}000 \tag{3.5}$$

Remediation: retool the injection-moulded housing (new mould insert, minimum 6-week lead time), re-certify the modified assembly, rework or scrap 50 units already built. At a unit BOM of \$480 and 50 units, scrap alone approaches \$24,000; the tooling and re-certification absorb the remainder.

3.4.3.2 Saving from Early Detection

$$\begin{aligned} \Delta C &= C_{\text{change}}(3) - C_{\text{change}}(1) \\ &= \$50{,}000 - \$500 \\ &= \$49{,}500 \end{aligned} \tag{3.6}$$

Using Equation 3.2 directly: $\Delta C(1, 3) = \$50\,(10^3 - 10^1) = \$50 \times 990 = \$49{,}500$.

3.4.3.3 Flaw B: MQTT Reconnection Loop Blocks Sensor Pipeline

The firmware's Wi-Fi reconnection routine holds a mutex that the sensor sampling task also requires. On reconnect, sensor sampling pauses for up to 3 seconds, during which items pass the camera unclassified.

Detected at the Design stage ($s = 1$, during prototype firmware testing with a simulated RF interference event):

$$C_{\text{change}}(1) = \$50 \times 10^1 = \$500 \tag{3.7}$$

Remediation: refactor the reconnection handler to operate on a dedicated task with its own stack, remove the shared mutex. Estimated effort: one sprint, two engineers, largely confined to the firmware module.

Detected at the Production stage ($s = 3$):

$$C_{\text{change}}(3) = \$50 \times 10^3 = \$50{,}000 \tag{3.8}$$

Remediation: firmware update requires regression testing of all embedded functionality, re-validation of the MQTT reliability specification across representative production-floor RF conditions, customer notification, potential SLA penalty under the HaaS contract.

Using Equation 3.2 for Flaw B: $\Delta C(1, 3) = \$50\,(10^3 - 10^1) = \$50 \times 990 = \$49{,}500$.

Both calculations reinforce the same conclusion: the most economically rational place to discover and fix design flaws is as early as possible. The Level 2 proof-of-concept prototype, built for a few hundred dollars in components and a few weeks of engineering time, is the instrument that makes that early discovery possible.

Try It Yourself

A custom motor-driver PCB has a layout error: the via drill size is incompatible with the contract manufacturer's minimum drill specification. The baseline fix cost at the Concept stage is $C_0 = \$80$ CAD.

1. Using $C(s) = C_0 \cdot 10^s$, compute the cost to fix this error at each of the four stages: Concept ($s = 0$), Design ($s = 1$), Build ($s = 2$), and Production ($s = 3$).
2. Using Equation 3.2, compute $\Delta C(1,\ 3)$ — the saving from catching the flaw at Design rather than Production.
3. Apply the decision flow in Figure 3.1 to the motor-driver PCB as a subsystem. Is it "novel or unprecedented"? "High uncertainty in performance"? "High consequence if it fails"? State a decision (Prototype or Specify-and-trust) with a one-sentence justification.

(Answers not provided — work through the arithmetic before moving on.)

3.4.3.3.1 In Practice. At the HaaS pricing of $1,200 CAD per unit and a gross margin of 60%, each unit contributes $720 CAD toward overhead and profit. A firmware flaw found at Production that requires reworking 50 units eliminates the entire gross margin contribution of 69 units — nearly three months of sales at the early-stage volumes typical of a twelve-person start-up. Prototype investment is not a cost; it is margin protection.

Common Pitfalls

3.4.3.3.2 Common Pitfalls.

Prototyping the easy subsystem first. It is tempting to begin with the conveyor belt or the power supply because those subsystems are well understood and progress feels fast. Resist this. Spend your first prototype sprint on the sort sensor pipeline. That is where your central hypothesis lives — whether a sub-$500 BOM can achieve 95% accuracy — and it is the only place where prototype results can confirm or refute the viability of the product. Comfort is not a valid criterion for prototype sequencing. Risk is.

Calling unplanned tinkering "iteration." Iteration is a documented cycle: you state a hypothesis, build the minimum artifact needed to test it, run the test, measure against a predefined criterion, and update the design based on what you learned. Tinkering — adjusting parameters without a recorded baseline, rerunning a test without changing the design, "trying things" without a clear success criterion — consumes time and produces no defensible knowledge. Every prototype run should be linked to a hypothesis from the specification document. If you cannot state the hypothesis in one sentence before you start the test, you are not iterating; you are tinkering.

Letting prototype quality inflate toward product quality. Once a prototype validates a subsystem, the pressure to "clean it up" before moving on is real. A clean, well-labelled PCB feels better than a breadboard. Resist the impulse unless a specific requirement — mechanical robustness, EMC pre-scan, thermal testing — demands it. The purpose of the Level 2 prototype is to answer the central question, not to produce a product. Over-finishing a prototype is borrowed time that the product phase will demand back.

Confusing specify-and-trust with specify-and-forget. A requirement written at the start of the project may cease to be correct if the design evolves. If your item mix changes and maximum item mass increases from 200 g to 350 g, revisit every OTS selection that was made under the original assumption. "Specify and trust" is not a permanent waiver; it is a standing decision that must be revisited when the requirements it was based on change.

Review Questions

1. **(Conceptual)** State the three criteria used to place a subsystem in the "Prototype it" column of the classification table. Give one example of a real subsystem that satisfies all three criteria, and one that satisfies none.
2. **(Conceptual)** The cost-of-change figure uses a logarithmic y-axis. Explain why a linear scale would be misleading for this data, and describe what the log-scale shape implies about where to invest prototype effort in a project timeline.
3. **(Applied — calculation)** The baseline fix cost for a firmware bug is $C_0 = \$120$ CAD.

   (a) Using $C(s) = C_0 \cdot 10^s$, compute the fix cost at all four stages.

(b) Using Equation 3.2, compute $\Delta C(1, 3)$.

(c) A triage decision moves the fix from Production ($s = 3$) back to Design ($s = 1$). Express the saving as a multiple of C_0.

4. **(Applied — multi-step)** The baseline fix cost is $C_0 = \$60$ CAD. A defect is discovered at the Build stage ($s = 2$).

   (a) Compute the cost to fix it at $s = 2$.

   (b) A redesign review triggers an earlier fix at $s = 1$ (Design). Compute the cost at $s = 1$.

   (c) Compute $\Delta C(1, 2)$ using Equation 3.2.

   (d) How many "baseline hours" (at C_0 each) does the earlier trigger save?

5. **(Applied — classification)** Using the decision flow in Figure 3.1, classify each of the following subsystems for a wearable ECG monitor. For each, answer the three questions (novel? uncertain? high consequence?) and state the decision with a one-line justification.

   (a) Off-the-shelf Bluetooth SoC (Nordic nRF52840, standard reference design).

   (b) Custom analog front-end for ECG signal acquisition.

   (c) Standard 500 mAh LiPo battery cell.

   (d) Custom dry electrode adhesive geometry (new material, no prior data).

   (e) USB-C charging circuit (TI BQ25619 reference design, datasheet verified).

End-of-Chapter Artifact: Subsystem Classification Table

The deliverable for this chapter is a completed subsystem classification table for your own project, structured identically to Table 3.2. It must contain one row per subsystem with the following columns:

Column	Content
Subsystem	Name of the subsystem
Novel?	Yes / Partial / No
Uncertain?	Yes / No
High-consequence?	Yes / No
Decision	Prototype *or* Specify & trust
Justification	One sentence explaining the decision

Every subsystem in your design must appear in the table. Subsystems classified as "Prototype" become candidates for the risk register in Chapter 4. Subsystems classified as "Specify & trust" must have a corresponding documented requirement and a named datasheet that confirms compliance.

For the sorting subsystem, the completed table is Table 3.2 above. The four prototype subsystems — sort sensor, divert actuator, MCU/firmware, and MQTT integration — will each receive a risk entry and a prototype hypothesis in Chapter 4. The three specify-and-trust subsystems — conveyor belt, PSU, and structural frame — will each receive a formal requirement and a datasheet citation at Milestone 1.

Summary and Transition

This chapter established the foundational judgment that separates effective from ineffective prototype practice: the decision of *what* to prototype.

A prototype optimizes for intrinsic value — knowledge and risk reduction. A product optimizes for extrinsic value — reliability, compliance, and lifecycle performance. Conflating the two wastes resources in both directions.

Selective prototyping directs effort at subsystems that are novel, uncertain, or high-consequence. Subsystems that fail all three tests are better handled through specify-and-trust: write a precise requirement, confirm it against a datasheet, and redirect the saved time to the subsystems that actually need experimentation.

The cost-of-change heuristic, $C_{\text{change}}(s) \approx C_0 \cdot 10^s$, quantifies why this matters. Finding a flaw at the Design stage rather than Production saves approximately $C_0 \times 990$ — for a $50 baseline, that is $49,500. For a twelve-person start-up operating on thin margins, early flaw detection is not an engineering nicety; it is a financial necessity.

Applied to the IoT sorting subsystem, the framework identifies four subsystems to prototype (sort sensor, divert actuator, MCU/firmware, MQTT integration) and three to specify-and-trust (conveyor belt, PSU, structural frame).

Transition to Chapter 4. With the subsystem classification complete, you are ready to formalize what you know and what you do not. Chapter 4 introduces the specification document — the Milestone 1 deliverable — and the risk register that will govern your prototype sequencing. Each of the four prototype subsystems will receive a formal hypothesis, a measurable success criterion, and a risk priority score. The three specify-and-trust subsystems will receive finalized requirements and sourcing decisions. The structure you have built in this chapter is the direct input to that process.

INDEX

C_0, baseline cost, 29

B2B, 7
B2C, 7
build-to-order, 7
business model, 6

commodity high-volume, 7
consequence criterion, 27
cost, 6
cost-of-change heuristic, 29

effort
    relative, 19
engineer, 4
extrinsic value, 27

fidelity, 19
firm size, 5

go/no-go, 18

hardware-as-a-service, 7

integration, 17
intrinsic value, 27
iteration
    controlled, 18
    uncontrolled, 18

lab track, 2
large enterprise, 5
licensing, 7

milestone, 3

novelty criterion, 27

off-the-shelf (OTS), 31
over-specification, 32

pitfall
    over-fidelity, 21
    prototype as mini-product, 21
product, 3, 26
product sale, 6
production volume, 6
project context brief, 10
project stage, 29
proof-of-concept, 1
prototype, 3, 25
    definition, 13
    functional, 19
    looks-like/works-like, 19
    pre-production, 19
    proof-of-concept, 18
Prototype Development Cycle, 13
prototype plan, 15

revenue model, 6
risk
    in prototyping, 14
    list, 14
running example, 8

selective prototyping, 27
seven-stage workflow, 2
SME, 5
specialty low-volume, 7
specification, 14
specify-and-trust, 27
start-up, 5
subsystem, 16

theory track, 2
time, 6

uncertainty criterion, 27
unit economics, 17

validation, 17